

Aula Detalhada - Datapath E Controle

Tema do dia: datapath, unidade de controle, PC, memória de instruções, banco de registradores, ULA, memória de dados, multiplexadores, sinais de controle, ciclo de execução, instruções tipo R, LOAD, STORE, BEQ e JUMP

Aula na sequência: 27

Objetivo: entender como o hardware executa instruções: por onde os dados passam no processador, quais blocos participam de cada tipo de instrução e como a unidade de controle gera sinais para coordenar o datapath.

1. Onde Esta Aula Entra No Estudo?

Na Aula 26, você aprendeu a olhar para uma instrução e separar seus campos:

```
opcode
rd
rs1
rs2
imediato
endereço/deslocamento
```

Agora a pergunta é:

```
como o processador usa esses campos para executar a instrução?
```

Exemplo:

```
ADD R1, R2, R3
```

A Aula 26 explica:

```
opcode = ADD
rd = R1
rs1 = R2
rs2 = R3
```

A Aula 27 explica:

1. o banco de registradores lê R2 e R3
2. a ULA soma os dois valores
3. o resultado volta para R1
4. a unidade de controle liga os sinais corretos

Essa aula é a ponte entre:

```
instrução em assembly
e
hardware executando sinais
```

2. Ideia Central

Um processador pode ser entendido em duas partes:

datapath
controle

Datapath:

caminho por onde os dados passam

Controle:

parte que decide quais caminhos ligar e quais operações fazer

Analogia:

datapath = estradas, registradores, ULA, memórias, MUXes
controle = semáforos e placas dizendo o que deve acontecer

Em termos digitais:

datapath carrega valores
controle gera sinais

3. O Que É Datapath?

Datapath é o conjunto de blocos que manipulam ou transportam dados dentro do processador.

Blocos típicos:

PC
memória de instruções
banco de registradores
ULA
memória de dados
somadores
multiplexadores
extensor de sinal
barramentos/fios

O datapath responde:

de onde o dado vem?
para onde o dado vai?
qual operação acontece no meio?
onde o resultado será guardado?

Exemplo em uma soma:

```
registradores -> ULA -> registrador destino
```

Exemplo em um LOAD:

```
registrador base + imediato -> ULA -> memória de dados -> registrador destino
```

4. O Que É Controle?

A unidade de controle observa a instrução e gera sinais.

Ela responde:

```
qual operação a ULA deve fazer?  
qual registrador deve ser escrito?  
a memória deve ser lida?  
a memória deve ser escrita?  
o dado escrito no registrador vem da ULA ou da memória?  
o PC vai para PC+4 ou para um alvo de desvio?
```

Controle não é o dado principal.

Controle é comando.

Exemplo:

```
opcode ADD
```

gera sinais como:

```
ALUControl = soma  
RegWrite = 1  
MemRead = 0  
MemWrite = 0
```

Exemplo:

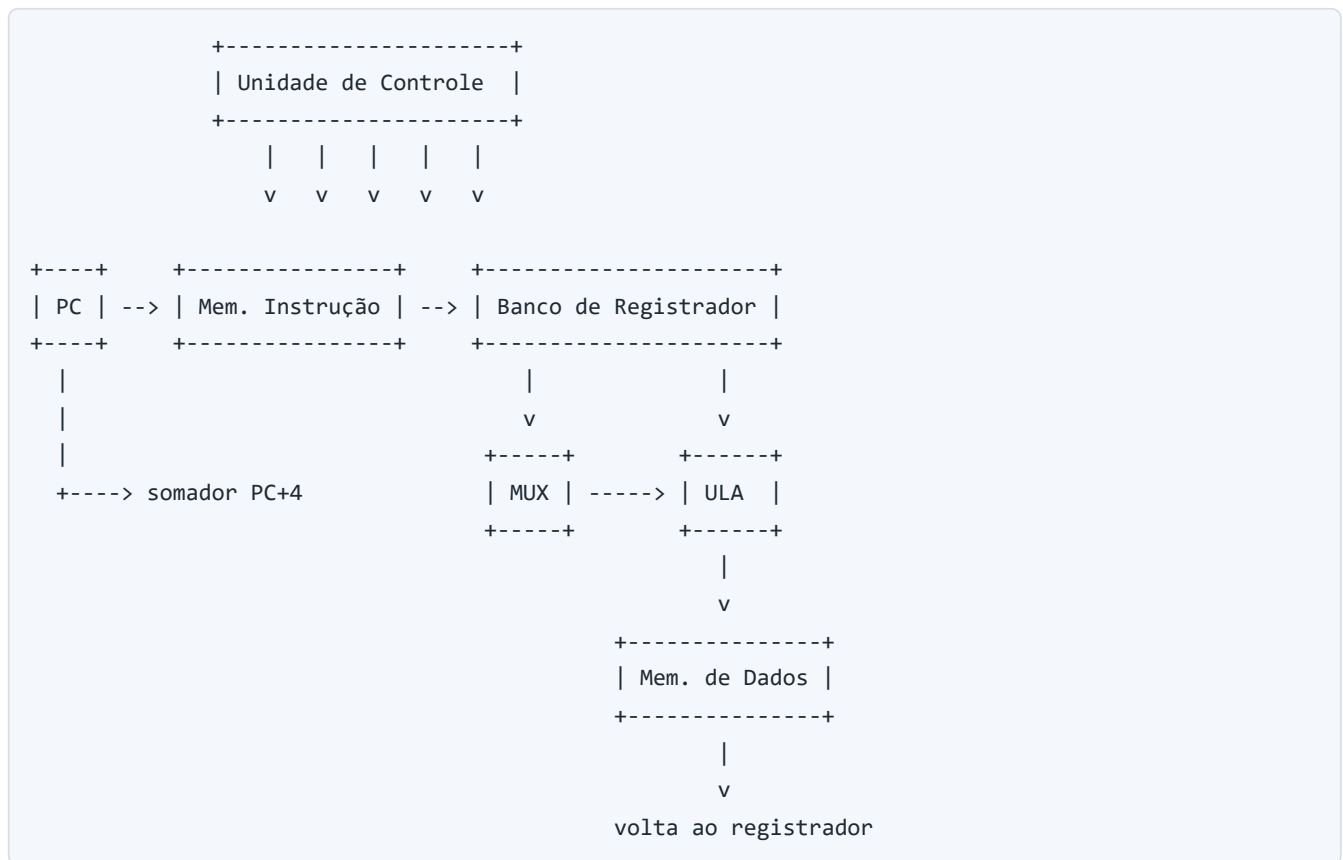
```
opcode LOAD
```

gera:

```
ALUControl = soma para calcular endereço  
MemRead = 1  
RegWrite = 1  
MemToReg = 1
```

5. Visão Geral De Um Datapath Simples

Um datapath didático pode ser visto assim:



Não precisa decorar o desenho exato de uma arquitetura específica.

Precisa entender os papéis.

6. Bloco PC

PC significa:

Program Counter

É o registrador que guarda o endereço da próxima instrução.

Fluxo normal:

PC aponta para instrução atual
instrução é buscada
PC é atualizado para próxima instrução

Em muitas arquiteturas:

$PC = PC + 4$

Por quê?

Porque a instrução pode ter 4 bytes.

Mas isso depende da arquitetura.

Para prova conceitual, memorize:

PC guarda o endereço da próxima instrução

Quando há desvio:

PC pode receber outro endereço

7. Memória De Instruções

Memória de instruções guarda o programa.

Entrada:

endereço vindo do PC

Saída:

instrução

Fluxo:

PC -> memória de instruções -> instrução

A instrução depois é separada em campos:

opcode
registradores
imediato
funct
endereços/deslocamentos

Essa etapa costuma ser chamada de:

fetch

Ou:

busca da instrução

8. Banco De Registradores

Banco de registradores é um conjunto de registradores acessados por número.

Exemplo:

```
R0, R1, R2, R3, ...
```

Ele normalmente permite:

```
ler dois registradores  
escrever um registrador
```

Por quê?

Porque muitas instruções usam dois operandos e um destino:

```
ADD R1, R2, R3
```

Leitura:

```
ler R2  
ler R3
```

Escrita:

```
escrever R1
```

Sinal importante:

```
RegWrite
```

Se:

```
RegWrite = 1
```

o banco de registradores escreve na borda de clock.

Se:

```
RegWrite = 0
```

não escreve.

9. ULA No Datapath

A ULA executa operações aritméticas e lógicas.

Operações comuns:

soma
subtração
AND
OR
XOR
comparação

Entradas:

operando A
operando B
sinal de controle da ULA

Saídas:

resultado
flags

No datapath, a ULA aparece em vários lugares:

somar operandos em ADD
subtrair em SUB
calcular endereço em LOAD/STORE
comparar valores em BEQ

Exemplo:

LOAD R1, [R2 + 8]

A ULA pode fazer:

endereço = $R2 + 8$

Ou seja:

ULA não serve só para ADD explícito

10. Memória De Dados

Memória de dados guarda variáveis e dados do programa.

Instruções que usam memória:

LOAD
STORE

LOAD:

memória -> registrador

STORE:

registrador -> memória

Sinais típicos:

MemRead
MemWrite

Para **LOAD**:

MemRead = 1
MemWrite = 0

Para **STORE**:

MemRead = 0
MemWrite = 1

Para **ADD**:

MemRead = 0
MemWrite = 0

11. Multiplexador No Datapath

MUX escolhe entre alternativas.

No datapath, MUX é essencial.

Por quê?

Porque diferentes instruções usam caminhos diferentes.

Exemplos de escolhas:

segundo operando da ULA vem de registrador ou imediato?
resultado escrito no registrador vem da ULA ou da memória?
próximo PC é PC+4 ou alvo de branch?
registrador destino vem de rd ou rt, em algumas arquiteturas?

Exemplo:

ADD R1, R2, R3

Segundo operando da ULA:

R3

Exemplo:

```
ADDI R1, R2, 5
```

Segundo operando da ULA:

imediato 5

O mesmo fio de entrada da ULA não pode receber os dois ao mesmo tempo.

Então usamos MUX.

Sinal:

ALUSrc

Se:

ALUSrc = 0 -> usa registrador

ALUSrc = 1 -> usa imediato

12. Extensor De Sinal

Imediatos costumam ter menos bits que os registradores.

Exemplo:

```
registradores de 32 bits  
imediato de 12 ou 16 bits
```

Antes de usar o imediato na ULA, ele pode precisar ser estendido.

Extensão de sinal:

preserva o sinal em complemento de 2

Exemplo com 4 bits para 8 bits:

```
0110 -> 00000110
```

positivo.

```
1010 -> 11111010
```

negativo em complemento de 2.

Extensão por zero:

preenche com zeros

Usada em algumas instruções lógicas/imediatas, dependendo da arquitetura.

Para prova conceitual:

extensor ajusta tamanho do imediato antes da ULA

13. Sinais De Controle Principais

Nomes variam por livro/arquitetura.

Mas estes são muito comuns em datapath didático:

Sinal	Função
RegWrite	habilita escrita no banco de registradores
ALUSrc	escolhe segundo operando da ULA: registrador ou imediato
ALUControl	define operação da ULA
MemRead	habilita leitura da memória de dados
MemWrite	habilita escrita na memória de dados
MemToReg	escolhe dado a escrever no registrador: ULA ou memória
Branch	indica instrução de desvio condicional
Jump	indica salto incondicional
PCSrc	escolhe próximo valor do PC

Você não precisa decorar todos os nomes exatos.

Precisa entender o papel:

controle escolhe caminhos e habilita escritas

14. Ciclo De Execução De Uma Instrução

Um ciclo conceitual de execução pode ser dividido em etapas:

1. Fetch
2. Decode
3. Execute
4. Memory
5. Writeback

Nem toda instrução usa todas as etapas do mesmo jeito.

14.1 Fetch

Busca a instrução.

```
instrução = MemInstr[PC]  
PC próximo = PC + 4
```

14.2 Decode

Decodifica a instrução.

```
separa opcode, registradores e imediato  
lê registradores fonte  
gera sinais de controle
```

14.3 Execute

Executa operação na ULA.

Pode ser:

```
soma  
subtração  
AND  
OR  
cálculo de endereço  
comparação de branch
```

14.4 Memory

Acessa memória de dados, se necessário.

Usado por:

```
LOAD  
STORE
```

14.5 Writeback

Escreve resultado no registrador destino, se necessário.

Usado por:

```
ADD  
SUB  
AND  
LOAD  
ADDI
```

Não usado por:

```
STORE
BEQ
JUMP
```

15. Execução De Uma Instrução Tipo R

Exemplo:

```
ADD R1, R2, R3
```

Objetivo:

```
R1 = R2 + R3
```

15.1 Caminho dos dados

```
PC -> memória de instruções
instrução -> banco de registradores
R2 e R3 -> ULA
resultado da ULA -> banco de registradores
```

15.2 Sinais de controle

Conceitualmente:

```
RegWrite = 1
ALUSrc = 0
ALUControl = ADD
MemRead = 0
MemWrite = 0
MemToReg = 0
Branch = 0
Jump = 0
```

Explicação:

```
RegWrite = 1 porque R1 será escrito
ALUSrc = 0 porque segundo operando vem de registrador
MemToReg = 0 porque resultado vem da ULA
memória de dados não é usada
```

16. Execução De ADDI

Exemplo:

```
ADDI R1, R2, 5
```

Objetivo:

```
R1 = R2 + 5
```

16.1 Caminho dos dados

```
R2 -> entrada A da ULA
imediato 5 -> extensor -> MUX -> entrada B da ULA
resultado -> R1
```

16.2 Sinais principais

```
RegWrite = 1
ALUSrc = 1
ALUControl = ADD
MemRead = 0
MemWrite = 0
MemToReg = 0
```

Diferença em relação ao `ADD` :

```
ADD usa dois registradores
ADDI usa registrador + imediato
```

17. Execução De LOAD

Exemplo:

```
LOAD R1, [R2 + 8]
```

Objetivo:

```
R1 = Mem[R2 + 8]
```

17.1 Caminho dos dados

```
R2 -> ULA
imediato 8 -> extensor -> ULA
ULA calcula endereço
memória de dados lê endereço
dado lido volta para R1
```

17.2 Sinais principais

```
RegWrite = 1
ALUSrc = 1
ALUControl = ADD
MemRead = 1
MemWrite = 0
MemToReg = 1
```

Explicação:

```
ALU soma base + deslocamento
MemRead lê memória
MemToReg escolhe dado vindo da memória
RegWrite grava em R1
```

Pegadinha:

```
em LOAD, a ULA normalmente calcula endereço
```

18. Execução De STORE

Exemplo:

```
STORE [R2 + 8], R1
```

Objetivo:

```
Mem[R2 + 8] = R1
```

18.1 Caminho dos dados

```
R2 -> ULA
imediato 8 -> extensor -> ULA
ULA calcula endereço
R1 fornece dado a escrever
memória grava R1 no endereço calculado
```

18.2 Sinais principais

```
RegWrite = 0
ALUSrc = 1
ALUControl = ADD
MemRead = 0
MemWrite = 1
```

Explicação:

```
STORE não escreve registrador
STORE escreve memória
```

Pegadinha:

```
MemWrite = 1
RegWrite = 0
```

19. Execução De BEQ

Exemplo:

```
BEQ R1, R2, alvo
```

Objetivo:

```
se R1 == R2, PC = alvo
senão, PC = PC + 4
```

19.1 Caminho dos dados

Uma forma comum:

```
R1 e R2 -> ULA
ULA faz subtração R1 - R2
se resultado é zero, R1 == R2
controle escolhe próximo PC
```

19.2 Sinais principais

```
RegWrite = 0
ALUSrc = 0
ALUControl = SUB
MemRead = 0
MemWrite = 0
Branch = 1
```

Se:

```
Branch = 1
Zero = 1
```

então:

```
PCSrc = 1
PC recebe alvo do branch
```

Se:

```
Zero = 0
```

então:

```
PCSrc = 0
PC recebe PC+4
```

20. Execução De JUMP

Exemplo:

```
JUMP alvo
```

Objetivo:

```
PC = alvo
```

Sinais conceituais:

```
Jump = 1
RegWrite = 0
MemRead = 0
MemWrite = 0
```

JUMP não precisa comparar registradores.

Ele simplesmente altera o fluxo.

21. Tabela Resumo De Controle

Tabela didática:

Instrução	RegWrite	ALUSrc	ALUControl	MemRead	MemWrite	MemToReg	Branch	Uso principal da ULA
ADD	1	0	ADD	0	0	0	0	somar registradores
SUB	1	0	SUB	0	0	0	0	subtrair registradores
ADDI	1	1	ADD	0	0	0	0	somar registrador + imediato
LOAD	1	1	ADD	1	0	1	0	calcular endereço
STORE	0	1	ADD	0	1	X	0	calcular endereço
BEQ	0	0	SUB	0	0	X	1	comparar registradores

X significa:

don't care

Ou seja:

o valor não importa para aquela instrução

22. Unidade De Controle Principal E Controle Da ULA

É comum dividir controle em duas partes:

controle principal
controle da ULA

Controle principal olha para:

opcode

e gera sinais gerais:

```
RegWrite
ALUSrc
MemRead
MemWrite
MemToReg
Branch
ALUOp
```

ALUOp é um resumo do tipo de operação.

Depois o controle da ULA usa:

```
ALUOp
funct, em algumas arquiteturas
```

para gerar:

```
ALUControl
```

Exemplo conceitual:

```
LOAD/STORE -> ALUOp diz "soma endereço"
BEQ         -> ALUOp diz "subtraia para comparar"
tipo R      -> ALUOp manda olhar campo funct
```

23. Controle Hardwired

Controle hardwired é controle feito com lógica combinacional/sequencial fixa.

Em vez de ter um programa interno de controle, o circuito decodifica a instrução diretamente.

Ideia:

```
opcode -> portas lógicas -> sinais de controle
```

Vantagens:

```
rápido
direto
bom para arquiteturas simples
```

Desvantagens:

```
mais difícil de alterar
pode ficar complexo em ISAs grandes
```

Para prova introdutória:

```
controle hardwired = controle por circuito lógico
```

24. Controle Microprogramado

Controle microprogramado usa uma memória de controle.

Ideia:

```
instrução -> seleciona microinstruções  
microinstruções -> geram sinais de controle
```

É como se instruções complexas fossem quebradas em passos internos menores.

Vantagens:

```
mais flexível  
facilita implementar instruções complexas
```

Desvantagens:

```
pode ser mais lento  
depende de memória de controle
```

Para esta prova, provavelmente basta saber a diferença conceitual:

```
hardwired -> lógica fixa  
microprogramado -> sequência de microinstruções
```

25. Datapath De Ciclo Único

No processador de ciclo único, cada instrução termina em um ciclo de clock.

Isso significa:

```
fetch  
decode  
execute  
memory  
writeback
```

acontecem dentro de um ciclo grande o suficiente para a instrução mais lenta.

Vantagem:

```
simples de entender
```

Desvantagem:

```
clock precisa ser longo o bastante para a pior instrução
```

Exemplo:

LOAD costuma ser mais lento que ADD

Porque LOAD precisa:

```
buscar instrução  
ler registrador  
calcular endereço  
ler memória de dados  
escrever registrador
```

Se tudo é ciclo único:

o clock precisa acomodar esse caminho longo

26. Datapath Multiciclo

No processador multiciclo, a instrução é quebrada em vários ciclos menores.

Exemplo:

```
ciclo 1 -> fetch  
ciclo 2 -> decode  
ciclo 3 -> execute  
ciclo 4 -> memory  
ciclo 5 -> writeback
```

Nem toda instrução usa todos os ciclos.

Exemplo:

ADD pode terminar antes de LOAD

Vantagens:

```
ciclo de clock menor  
reuso de blocos em ciclos diferentes
```

Desvantagens:

```
controle mais complexo  
precisa de FSM de controle
```

Conexão com aulas anteriores:

FSM pode controlar etapas internas do processador

27. Controle Como FSM

Em datapath multiciclo, o controle pode ser uma máquina de estados.

Estados possíveis:

```
FETCH
DECODE
EXECUTE_R
EXECUTE_MEM
MEM_READ
MEM_WRITE
WRITEBACK
BRANCH
```

Exemplo de sequência para **ADD** :

```
FETCH -> DECODE -> EXECUTE_R -> WRITEBACK -> FETCH
```

Exemplo de sequência para **LOAD** :

```
FETCH -> DECODE -> EXECUTE_MEM -> MEM_READ -> WRITEBACK -> FETCH
```

Exemplo de sequência para **STORE** :

```
FETCH -> DECODE -> EXECUTE_MEM -> MEM_WRITE -> FETCH
```

Exemplo de sequência para **BEQ** :

```
FETCH -> DECODE -> BRANCH -> FETCH
```

Isso mostra por que FSM é importante.

O controle do processador pode ser literalmente uma FSM.

28. Pipeline Em Nível Conceitual

Pipeline divide a execução em estágios.

Exemplo clássico:

```
IF -> busca da instrução
ID -> decodificação/leitura de registradores
EX -> execução/ULA
MEM -> acesso à memória
WB -> escrita de volta
```

Enquanto uma instrução está em **EX** , outra pode estar em **ID** , outra em **IF** .

Ideia:

aumentar vazão

Não significa que uma instrução individual fica necessariamente mais rápida.

Significa que várias instruções podem estar em andamento ao mesmo tempo.

Para esta aula, basta o conceito:

pipeline sobrepõe etapas de instruções diferentes

29. Hazards De Pipeline

Se pipeline aparecer em prova conceitual, três riscos podem ser citados:

hazard estrutural
hazard de dados
hazard de controle

29.1 Hazard estrutural

Ocorre quando duas instruções precisam do mesmo recurso ao mesmo tempo.

Exemplo:

duas etapas querendo a mesma memória

29.2 Hazard de dados

Ocorre quando uma instrução depende do resultado de outra ainda não concluída.

Exemplo:

```
ADD R1, R2, R3
SUB R4, R1, R5
```

SUB precisa de R1, que está sendo produzido por ADD.

29.3 Hazard de controle

Ocorre por causa de desvios.

Exemplo:

```
BEQ R1, R2, alvo
```

Enquanto não se sabe se desvia, o processador pode não saber qual instrução buscar.

Para o edital, isso pode ser mais do que necessário, mas ajuda a conectar com controle.

30. Rastreio Completo: ADD

Estado inicial:

```
R2 = 7
R3 = 5
```

Instrução:

```
ADD R1, R2, R3
```

Passos:

1. PC busca a instrução
2. controle identifica opcode ADD
3. banco de registradores lê R2=7 e R3=5
4. controle manda ULA somar
5. ULA calcula $7+5=12$
6. RegWrite=1 grava 12 em R1
7. PC vai para próxima instrução

Resultado:

```
R1 = 12
```

31. Rastreio Completo: LOAD

Estado inicial:

```
R2 = 100
Mem[108] = 55
```

Instrução:

```
LOAD R1, [R2 + 8]
```

Passos:

1. PC busca a instrução
2. controle identifica LOAD
3. banco de registradores lê R2=100
4. imediato 8 é estendido
5. ALUSrc=1 escolhe imediato
6. ULA calcula endereço $100+8=108$
7. MemRead=1 lê Mem[108]=55
8. MemToReg=1 escolhe dado da memória
9. RegWrite=1 grava 55 em R1

Resultado:

R1 = 55

32. Rastreio Completo: STORE

Estado inicial:

R1 = 77
R2 = 200

Instrução:

STORE [R2 + 12], R1

Passos:

1. PC busca a instrução
2. controle identifica STORE
3. banco de registradores lê R2=200 e R1=77
4. imediato 12 é estendido
5. ULA calcula endereço 200+12=212
6. MemWrite=1 grava 77 em Mem[212]
7. RegWrite=0, pois nenhum registrador recebe resultado

Resultado:

Mem[212] = 77

33. Rastreio Completo: BEQ

Estado inicial:

R1 = 9
R2 = 9
PC = 1000
alvo = 1040

Instrução:

BEQ R1, R2, alvo

Passos:

1. PC busca a instrução
2. controle identifica BEQ
3. banco de registradores lê R1=9 e R2=9
4. ULA faz $R1 - R2 = 0$
5. flag Zero = 1
6. Branch = 1 e Zero = 1
7. PC recebe alvo

Resultado:

PC = 1040

Se R1 fosse diferente de R2 :

Zero = 0
PC = PC + 4

34. Como Questões Costumam Cobrar

As questões tendem a pedir:

identificar o papel do datapath
identificar o papel da unidade de controle
dizer quais blocos uma instrução usa
marcar sinais de controle de ADD/LOAD/STORE/BEQ
diferenciar RegWrite e MemWrite
diferenciar MemRead e MemWrite
dizer quando ALUSrc usa imediato
dizer quando MemToReg escolhe memória
calcular endereço efetivo de LOAD/STORE
explicar como branch muda o PC
comparar ciclo único e multiciclo
reconhecer controle como FSM

Exemplo:

Em STORE, RegWrite deve ser 0 ou 1?

Resposta:

0, porque STORE escreve memória, não registrador.

Outro:

Em LOAD, MemToReg deve selecionar ULA ou memória?

Resposta:

memória, porque o dado lido da memória será escrito no registrador.

35. Erros Comuns

35.1 Achar que controle carrega dados

Controle não carrega operandos principais.

Quem carrega dados é o datapath.

Controle apenas diz:

```
ligue este MUX
escreva este registrador
faça soma na ULA
leia memória
```

35.2 Confundir RegWrite com MemWrite

RegWrite:

```
escreve no banco de registradores
```

MemWrite:

```
escreve na memória de dados
```

STORE:

```
RegWrite = 0
MemWrite = 1
```

35.3 Achar que LOAD não usa ULA

LOAD usa ULA para calcular endereço:

```
base + deslocamento
```

35.4 Achar que BEQ salva resultado da subtração

BEQ pode usar a ULA para comparar.

Mas não grava o resultado da subtração em registrador.

Ele usa a flag `Zero` para decidir o PC.

35.5 Confundir memória de instruções com memória de dados

Memória de instruções:

guarda programa

Memória de dados:

guarda valores do programa

Em arquiteturas reais isso pode variar, mas em datapath didático essa separação ajuda muito.

35.6 Achar que ciclo único significa uma etapa só

Ciclo único significa:

uma instrução termina em um ciclo de clock

Mas dentro desse ciclo, ela ainda passa por vários blocos.

36. Exercícios

36.1 Conceituais

1. O que é datapath?
1. O que é unidade de controle?
1. Qual é a função do PC?
1. Qual é a função da memória de instruções?
1. Qual é a função do banco de registradores?
1. Qual é a função da ULA no datapath?
1. Qual é a diferença entre memória de instruções e memória de dados?
1. Para que serve um MUX no datapath?
1. O que faz o sinal `RegWrite` ?
1. O que faz o sinal `MemWrite` ?
1. O que faz o sinal `MemRead` ?
1. O que faz o sinal `ALUSrc` ?
1. O que faz o sinal `MemToReg` ?
1. Por que `LOAD` usa a ULA?
1. Por que `BEQ` pode usar subtração na ULA?
1. Qual é a diferença entre ciclo único e multiciclo?
1. Por que o controle multiciclo pode ser implementado como FSM?

1. O que é pipeline em nível conceitual?

36.2 Controle Por Instrução

Para cada instrução, indique os sinais principais.

1. `ADD R1, R2, R3`: `RegWrite`, `ALUSrc`, `MemRead`, `MemWrite`, `MemToReg`.
1. `ADDI R1, R2, 5`: `RegWrite`, `ALUSrc`, `MemRead`, `MemWrite`, `MemToReg`.
1. `LOAD R1, [R2 + 8]`: `RegWrite`, `ALUSrc`, `MemRead`, `MemWrite`, `MemToReg`.
1. `STORE [R2 + 8], R1`: `RegWrite`, `ALUSrc`, `MemRead`, `MemWrite`.
1. `BEQ R1, R2, alvo`: `RegWrite`, `ALUSrc`, `MemRead`, `MemWrite`, `Branch`.

36.3 Rastreio

Considere:

```
R1 = 4
R2 = 10
R3 = 100
Mem[108] = 33
```

1. Após `ADD R4, R1, R2`, qual é `R4`?
1. Após `LOAD R5, [R3 + 8]`, qual é `R5`?
1. Após `STORE [R3 + 12], R2`, qual posição de memória é alterada e qual valor é gravado?
1. Em `BEQ R1, R2, alvo`, o branch é tomado?
1. Em `BEQ R1, R1, alvo`, o branch é tomado?

36.4 Múltipla Escolha

1. Em uma instrução `LOAD`, o dado escrito no registrador vem:

- A) sempre do PC.
- B) da memória de dados.
- C) da memória de instruções.
- D) do sinal Branch.

1. Em uma instrução `STORE`, o sinal que deve ser ativado é:

- A) `MemWrite`.
- B) `MemToReg`.
- C) `Jump`.
- D) `RegWrite`.

1. O sinal `ALUSrc` escolhe:

- A) se a memória será lida ou escrita.
- B) se o segundo operando da ULA vem de registrador ou imediato.
- C) se o PC existe.

D) quantos registradores há no processador.

1. Em `BEQ`, se a ULA compara dois registradores iguais, a flag mais relevante é:

A) `Zero`.

B) `MemWrite`.

C) `ALUSrc`.

D) `VDD`.

1. Datapath é melhor descrito como:

A) apenas o programa escrito em C.

B) o caminho físico/lógico por onde dados passam no processador.

C) um tipo de memória DRAM.

D) o arquivo de texto do assembly.

1. Unidade de controle é melhor descrita como:

A) a parte que gera sinais para coordenar o datapath.

B) o local onde todos os dados de usuário são armazenados.

C) a tensão de alimentação.

D) o registrador que sempre guarda o resultado final.

1. Em um processador de ciclo único, o período de clock precisa acomodar:

A) apenas a instrução mais rápida.

B) apenas o tempo do PC.

C) a instrução mais lenta/caminho mais longo.

D) nenhuma instrução.

37. Gabarito

1. É o conjunto de blocos por onde os dados passam e são manipulados no processador.

1. É a parte que decodifica a instrução e gera sinais para coordenar o datapath.

1. Guardar o endereço da próxima instrução.

1. Guardar o programa e entregar a instrução indicada pelo PC.

1. Guardar registradores, permitir leituras de operandos e escrita de resultados.

1. Executar operações aritméticas/lógicas, calcular endereços e ajudar em comparações.

1. Memória de instruções guarda programa; memória de dados guarda valores manipulados pelo programa.

1. Escolher entre caminhos/entradas alternativas, como registrador ou imediato.

1. Habilita escrita no banco de registradores.

1. Habilita escrita na memória de dados.

1. Habilita leitura da memória de dados.

1. Escolhe se o segundo operando da ULA vem do registrador ou do imediato.

1. Escolhe se o valor escrito no registrador vem da ULA ou da memória.

1. Para calcular o endereço efetivo, como base + deslocamento.

1. Porque subtrair dois valores iguais gera zero; a flag `Zero` indica igualdade.
1. Ciclo único termina cada instrução em um ciclo longo; multiciclo divide a instrução em vários ciclos menores.
1. Porque cada etapa da execução pode ser um estado, como `FETCH`, `DECODE`, `EXECUTE`, `MEMORY` e `WRITEBACK`.
1. É a sobreposição de etapas de instruções diferentes para aumentar vazão.
1. `RegWrite=1`, `ALUSrc=0`, `MemRead=0`, `MemWrite=0`, `MemToReg=0`.
1. `RegWrite=1`, `ALUSrc=1`, `MemRead=0`, `MemWrite=0`, `MemToReg=0`.
1. `RegWrite=1`, `ALUSrc=1`, `MemRead=1`, `MemWrite=0`, `MemToReg=1`.
1. `RegWrite=0`, `ALUSrc=1`, `MemRead=0`, `MemWrite=1`.
1. `RegWrite=0`, `ALUSrc=0`, `MemRead=0`, `MemWrite=0`, `Branch=1`.
1. `R4 = 4 + 10 = 14`.
1. Endereço: `R3 + 8 = 100 + 8 = 108`. Portanto, `R5 = Mem[108] = 33`.
1. Endereço: `R3 + 12 = 112`. Grava `R2 = 10` em `Mem[112]`.
1. Não. `R1=4` e `R2=10`, então não são iguais.
1. Sim. `R1` é igual a `R1`.
1. B.
1. A.
1. B.
1. A.
1. B.
1. A.
1. C.

38. O Que Memorizar

Memorize:

```
datapath -> caminho dos dados
controle -> sinais que comandam o caminho
```

Memorize os blocos:

```
PC -> endereço da próxima instrução
memória de instruções -> entrega instrução
banco de registradores -> lê/escreve registradores
ULA -> opera e calcula endereços
memória de dados -> LOAD/STORE
MUX -> escolhe caminhos
```

Memorize sinais:

RegWrite -> escreve registrador
MemWrite -> escreve memória
MemRead -> lê memória
ALUSrc -> registrador ou imediato na ULA
MemToReg -> ULA ou memória para escrever no registrador
Branch -> desvio condicional
Jump -> salto incondicional

Memorize por instrução:

ADD -> lê 2 registradores, ULA, escreve registrador
ADDI -> lê registrador, usa imediato, ULA, escreve registrador
LOAD -> ULA calcula endereço, lê memória, escreve registrador
STORE -> ULA calcula endereço, escreve memória
BEQ -> ULA compara, PC pode mudar
JUMP -> PC muda sem comparar

Pegadinha mais importante:

LOAD usa MemRead e RegWrite
STORE usa MemWrite e não usa RegWrite

Outra pegadinha:

BEQ usa ULA para comparar, mas não grava resultado da ULA em registrador

39. Plano De Estudo Para Esta Aula

Tempo sugerido: **50 a 80 minutos**.

39.1 Primeira passada

Leia com foco em:

datapath vs controle
PC
banco de registradores
ULA
memória de dados
MUX
sinais de controle

39.2 Segunda passada

Ref faça sem olhar:

```
caminho do ADD  
caminho do LOAD  
caminho do STORE  
caminho do BEQ
```

39.3 Exercícios prioritários

Se tiver pouco tempo, faça:

```
9, 10, 12, 13, 14, 15, 19, 21, 22, 23, 25, 26, 29, 30, 31 e 32
```

Eles cobrem os sinais e caminhos mais prováveis.

40. Conexão Com A Revisão Final

Esta aula fecha o bloco atual de conteúdo planejado.

Agora você consegue enxergar a cadeia completa:

```
bits  
portas  
circuitos combinacionais  
circuitos sequenciais  
registradores  
ULA  
memórias  
instruções  
datapath  
controle
```

Para revisar Arquitetura de Computadores, conecte:

```
Aula 25 -> algoritmo, assembly e operandos  
Aula 26 -> formato de instrução e modos de endereçamento  
Aula 27 -> execução no datapath e sinais de controle
```

Resumo final:

```
programa vira instruções  
instruções têm campos  
campos alimentam controle  
controle comanda datapath  
datapath move e transforma dados
```

Esse é o coração de organização de computadores em nível de prova objetiva.